

Study 프로젝트 — 필수 백엔드 지식 정리

본 문서는 앞선 [backend-data-flow.pdf](#) 의 코드 흐름을 이해하기 위해 실제로 알아야 하는 백엔드 핵심 개념만 골라 정리한 학습용 자료입니다.

대상: Spring Boot 4.x · Java 17 · Spring Data JPA · MariaDB · OAuth 2.0 · 카카오페이 · JavaMail.

목차

1. Spring Boot 기본 — 진입점·자동 구성·DI
2. 웹 계층 — @RestController, ResponseEntity, DTO, 검증
3. Jackson JSON 매핑 (@JsonProperty)
4. 서비스 계층과 트랜잭션 (@Transactional)
5. 영속성 — JPA · JpaRepository · @Query · MyBatis
6. 엔티티 매핑과 라이프사이클 콜백
7. 데이터베이스 — MariaDB · 인덱스 · ddl-auto
8. 세션 기반 인증 · HttpSession · 쿠키
9. 비밀번호 보안 — BCrypt · SecureRandom · 재설정 토큰
10. CORS — 출처/메서드/자격 증명
11. OAuth 2.0 — Authorization Code · state · CSRF
12. 외부 API 호출 — RestClient (vs RestTemplate/WebClient)
13. 카카오페이 단건 결제 일반론 (ready / approve)
14. 이메일 — JavaMailSender · SMTP · MimeMessage
15. 비동기와 스케줄링 — @EnableAsync · @Scheduled · Cron
16. 예외 처리 — @RestControllerAdvice · 전역 핸들러
17. OpenAPI / Swagger UI — springdoc
18. 환경 설정 — application.properties · @Value · 환경변수
19. 학습 체크리스트 / 추천 자료

1. Spring Boot 기본

1-1. 어플리케이션 진입점

`@SpringBootApplication` 한 어노테이션이 사실은 세 개를 합친 것입니다.

- `@Configuration` — 자바 기반 빈 정의 클래스.
- `@EnableAutoConfiguration` — classpath 의존성을 보고 자동 구성 활성화 (예: `spring-boot-starter-web` 있으면 톰캣/디스패처서블릿 자동 구성).
- `@ComponentScan` — 현재 패키지 이하의 `@Component/@Service/@Repository/@Controller` 빈 자동 등록.

```
@SpringBootApplication
@EnableAsync           // 비동기 메일 발송용
@EnableScheduling      // 매일 20시 일일 메일 스케줄러용
public class StudyApplication {
    public static void main(String[] args) {
        SpringApplication.run(StudyApplication.class, args);
    }
}
```

1-2. 의존성 주입(DI) / IoC 컨테이너

- 스프링 컨테이너가 빈(bean)을 만들고 라이프사이클을 관리합니다.
- 주입 방식은 생성자 주입을 표준으로 사용합니다 (불변 + 테스트 용이 + 순환 의존성 탐지).

- Lombok `@RequiredArgsConstructor` 로 final 필드 생성자를 자동 생성.

```
@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository userRepository; // 자동 주입
    private final PasswordEncoder passwordEncoder; // 자동 주입
    private final EmailVerificationService emailService; // 자동 주입
}
```

1-3. 빈 스코프 / 라이프사이클(필수만)

스코프	의미	예
singleton (기본)	컨테이너당 1개 — 대부분의 서비스/리포지토리	AuthService
prototype	주입 때마다 새 인스턴스	특수 케이스
request / session	웹 요청·세션 단위	로그인 컨텍스트

2. 웹 계층

2-1. 컨트롤러 어노테이션

어노테이션	역할
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> . 메서드 반환값이 자동으로 JSON 직렬화.
<code>@RequestMapping("/api/...")</code>	클래스/메서드 단위 URL 매핑.
<code>@GetMapping</code> 등	HTTP 메서드별 단축형.
<code>@PathVariable</code>	URL 경로 변수: <code>/notices/{id}</code> .
<code>@RequestParam</code>	쿼리/폼 파라미터: <code>?page=0&size=10</code> .
<code>@RequestBody</code>	요청 본문 JSON → 객체 역직렬화.
<code>@RequestHeader</code>	헤더 값 주입 (예: <code>X-Admin-Token</code>).

2-2. ResponseEntity

상태 코드/헤더/바디를 명시적으로 제어해야 할 때 사용합니다.

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(new UserResponse(...));

// 302 redirect (카카오페이 approve 후 프론트로)
return ResponseEntity.status(HttpStatus.FOUND)
    .location(URI.create(frontendUrl + "/pay/success?...")).build();
```

2-3. DTO와 Bean Validation

- 엔티티를 직접 노출하지 말고 **request/response DTO**를 분리합니다 — 보안(비밀번호 해시 노출 방지) + 계약 분리.
- `@Valid` + Jakarta Validation 어노테이션으로 입력 검증을 컨트롤러에서 일괄 처리.

```
public record SignupRequest(
    @NotBlank @Size(min=4, max=20) String username,
```

```

    @NotBlank @Size(min=8) String password,
    @Email @NotBlank String email
) {}

@PostMapping("/api/auth/signup")
public UserResponse signup(@Valid @RequestBody SignupRequest req) { ... }

```

검증 실패 시 `MethodArgumentNotValidException` 이 발생 → 전역 핸들러에서 400 으로 변환.

3. Jackson JSON 매핑

스프링은 기본적으로 Jackson 으로 JSON ↔ 객체 변환합니다. 필드 이름이 외부 API 의 `snake_case` 와 다를 때 `@JsonProperty` 로 명시 매핑합니다 (커밋 `9357017` 의 PlaceSearchResponse 가 이 케이스).

```

public class PlaceItem {
    @JsonProperty("place_name")    private String name;
    @JsonProperty("road_address_name") private String roadAddress;
    @JsonProperty("x")             private String lng;    // 경도
    @JsonProperty("y")             private String lat;    // 위도
}

```

- `@JsonIgnore` — 직렬화/역직렬화에서 제외.
- `@JsonInclude(NON_NULL)` — null 필드 생략.
- `@JsonFormat(pattern="yyyy-MM-dd HH:mm:ss")` — 날짜 포맷.

4. 서비스 계층과 트랜잭션

4-1. 책임 분리

- Controller : HTTP 입출력만 — 검증/세션/응답 코드.
- Service : 비즈니스 규칙 + 트랜잭션 경계 + 외부 API 호출 조립.
- Repository : DB 접근만 (CRUD/쿼리).

4-2. @Transactional 핵심

속성	의미
전파(propagation)	기본 <code>REQUIRED</code> — 기존 트랜잭션 있으면 참여, 없으면 새로 시작.
격리(isolation)	기본은 DB 기본값(MariaDB 는 <code>REPEATABLE_READ</code>).
readOnly	읽기 전용 — 쓰기 쿼리 차단 + 성능 힌트.
rollbackFor	기본은 <code>RuntimeException</code> + <code>Error</code> 만 롤백. checked Exception 은 명시해야 롤백.

주의 — 자기 호출 함정: 같은 빈 안의 메서드를 그냥 `this.method()` 로 호출하면 `@Transactional` 이 적용되지 않습니다. 스프링이 프록시로 트랜잭션을 걸기 때문에, 외부에서 진입할 때만 동작합니다.

5. 영속성 — JPA · JpaRepository

5-1. JPA / Hibernate / Spring Data JPA

- **JPA**: 자바 ORM 표준 스펙 (인터페이스 모음).
- **Hibernate**: JPA 의 가장 널리 쓰이는 구현체. Spring Boot 기본.
- **Spring Data JPA**: Repository 인터페이스만 정의하면 구현체를 자동 생성해 주는 추상화 계층.

5-2. JpaRepository 의 3가지 쿼리 작성법

```
public interface NoticeRepository extends JpaRepository<Notice, Long> {
    // ① 메서드 명명 규약 - 메서드 이름이 곧 쿼리
    Page<Notice> findByTitleContainingOrContentContaining(
        String t, String c, Pageable pageable);

    // ② @Query - JPQL 직접 작성
    @Query("select count(c) from Comment c where c.noticeId = :id")
    long countCommentsByNoticeId(@Param("id") Long id);

    // ③ 네이티브 SQL
    @Query(value = "select * from notice where match(title,content) against(?1)",
        nativeQuery = true)
    List<Notice> fullTextSearch(String keyword);
}
```

5-3. Pageable / Page

```
// 컨트롤러
public Page<NoticeListItem> list(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size) {
    Pageable pageable = PageRequest.of(page, size, Sort.by("createdAt").descending());
}
```

```
return service.search(type, keyword, pageable);
}
```

Page<T> 는 content, totalElements, totalPages, number, size 를 그대로 JSON 으로 반환합니다.

5-4. N+1 문제 (게시판 같은 1:N 에서 빈번)

게시글 100건을 가져온 뒤 각 글마다 댓글 수 SELECT → 총 101 쿼리. 해결책:

- 배치 조회: 게시글 ID 들을 모아 WHERE noticeId IN (...) 1회 (이 프로젝트의 enrich() 패턴).
- fetch join: @Query("select n from Notice n join fetch n.comments") .
- @EntityGraph .

5-5. MyBatis

JPA 와 함께 쓰는 SQL 매핑 프레임워크. XML 또는 어노테이션으로 SQL 을 직접 작성. 복잡한 통계/리포트성 쿼리에 유리. 이 프로젝트는 스타터만 포함되어 있고 활성 매핑은 없습니다.

6. 엔티티 매핑과 라이프사이클

```
@Entity
@Table(name = "notice",
        indexes = @Index(name = "idx_notice_created", columnList = "created_at"))
@Getter @Setter @NoArgsConstructor
public class Notice {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String content;

    @Column(name = "view_count", nullable = false)
    private int viewCount;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    void onCreate() { createdAt = updatedAt = LocalDateTime.now(); }

    @PreUpdate
    void onUpdate() { updatedAt = LocalDateTime.now(); }
}
```

어노테이션	역할
@Entity / @Table	DB 테이블 매핑.
@Id / @GeneratedValue	PK · 생성 전략(IDENTITY/SEQUENCE/UUID).
@Column	컬럼 속성: nullable, length, unique, name 등.
@Enumerated(EnumType.STRING)	Enum → 문자열 컬럼 저장 (예: Payment.status).

@PrePersist / @PreUpdate	저장/수정 직전 콜백 (생성·수정 시간 자동 채움).
@ManyToOne / @OneToMany	연관관계 (이 프로젝트는 ID 기반 느슨한 연관을 선호).

7. 데이터베이스 — MariaDB

- JDBC URL : `jdbc:mariadb://localhost:3306/notice` . 드라이버 `org.mariadb.jdbc.Driver` .
- HikariCP 가 기본 커넥션 풀(부트가 자동 설정).
- ddl-auto** 설정값:
 - `none` — 아무것도 안 함 (운영 권장).
 - `validate` — 매핑 검증만.
 - `update` — 차이가 있으면 ALTER (이 프로젝트). 데이터 손실은 안 나지만, 컬럼 삭제는 안 되고 씀
 - `create / create-drop` — 매번 재생성 (개발 초기/테스트만).
- 인덱스**: 자주 조회되는 컬럼(`email`, `kakao_id`, (`notice_id`, `created_at`))에는 인덱스를 명시. UNIQUE 제약은 비즈니스 무결성 + 인덱스 효과 동시.

8. 세션 기반 인증

8-1. 토큰 vs 세션

	세션 (이 프로젝트)	JWT 등 토큰
상태	서버에 사용자 상태 보관	서버 무상태(stateless)
인증 매개체	<code>JSESSIONID</code> 쿠키	<code>Authorization: Bearer ...</code>
로그아웃	<code>session.invalidate()</code> 즉시 무효	토큰 만료까지 살아있음(블랙리스트 필요)
스케일	여러 인스턴스 → 세션 클러스터/Redis 필요	스케일 용이

8-2. HttpSession 사용 패턴

```
// 로그인 성공 시
session.setAttribute(AuthController.SESSION_USER_KEY, user.getId());

// 모든 보호 컨트롤러에서
Long userId = (Long) session.getAttribute(AuthController.SESSION_USER_KEY);
if (userId == null) throw new RuntimeException(HttpStatus.UNAUTHORIZED);
```

8-3. 쿠키 보안

- `HttpOnly` : JS 에서 쿠키 접근 차단 → XSS 로 세션 ID 탈취 방지.
- `Secure` : HTTPS 에서만 전송.
- `SameSite=Lax/Strict/None` : CSRF 완화. 이 프로젝트는 `Lax` .

9. 비밀번호 보안 — BCrypt

비밀번호는 절대 평문 저장 금지. 단방향 해시 + Salt 필수.

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(); // 기본 강도 10
}

// 저장
```

```
String hash = passwordEncoder.encode(rawPassword);

// 검증
boolean ok = passwordEncoder.matches(rawPassword, user.getPassword());
```

- BCrypt 는 Salt 가 해시 문자열에 포함되므로 **같은 비밀번호도 매번 다른 결과**가 나옵니다.
- 강도(round) 가 1 올라갈 때마다 연산량 2배 → 무차별 대입 비용 증가.
- 비밀번호 재설정 토큰은 `SecureRandom` + URL-safe Base64 32바이트, 만료시간(예: 30분), 1회용으로 운영.

10. CORS

브라우저는 **다른 출처(origin = scheme + host + port)** 로의 JS 요청을 기본 차단합니다. 백엔드가 응답 헤더로 허용을 명시해야 합니다.

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry r) {
        r.addMapping("/api/**")
            .allowedOrigins("http://localhost:5173", "http://localhost:5174")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowCredentials(true); // 쿠키(세션) 동봉 허용
    }
}
```

주의: `allowCredentials=true` 면 `allowedOrigins` 에 와일드카드 `*` 를 못 씁니다. 정확한 출처를 나열하거나 `allowedOriginPatterns` 를 사용하세요.

11. OAuth 2.0 — Authorization Code Flow

11-1. 흐름

- ① 사용자가 "카카오로 로그인" 클릭
→ 브라우저를 인가 서버로 redirect
GET kauth.kakao.com/oauth/authorize?
client_id=...&redirect_uri=...&response_type=code
&state=xxx (CSRF 방지, 네이버/구글 필수)
- ② 사용자가 인가 서버에서 로그인 + 동의
- ③ 인가 서버가 우리 서버 callback 으로 redirect
GET /api/auth/kakao/callback?code=ABC&state=xxx
- ④ 서버가 code 를 access_token 으로 교환 (서버↔서버)
POST kauth.kakao.com/oauth/token
grant_type=authorization_code & code=ABC & client_secret=...
- ⑤ access_token 으로 사용자 프로필 조회
GET kapi.kakao.com/v2/user/me Authorization: Bearer ...
- ⑥ users 테이블에 upsert + 세션 부여

11-2. 왜 code 를 한 번 더 교환할까?

- access_token 이 URL/브라우저 히스토리/로그에 노출되지 않게 하려고.
- code 는 1회용 + 짧은 만료 → 탈취돼도 재사용 불가 (이 프로젝트는 KAKAO_LAST_CODE 로 재사용 추가 차단).

11-3. state 파라미터 (CSRF 방어)

로그인 요청 시 랜덤 문자열을 세션에 저장, 콜백에서 동일한지 검증. 공격자가 임의로 callback URL 을 호출해도 state 가 다르면 거부 → 피해자의 계정에 공격자 소셜계정 연결되는 공격을 막습니다.

12. 외부 API 호출 — RestClient

Spring 6.1+ 에서 도입된 동기 HTTP 클라이언트. `RestTemplate` 의 후속이며 `WebClient` 와 비슷한 fluent API.

```
RestClient client = RestClient.builder()
    .baseUrl("https://dapi.kakao.com")
    .defaultHeader(HttpHeaders.AUTHORIZATION, "KakaoAK " + apiKey)
    .build();

PlaceSearchResponse res = client.get()
    .uri(uri -> uri.path("/v2/local/search/keyword.json")
        .queryParams("query", query)
        .queryParams("page", page)
        .build())
    .retrieve()
    .body(PlaceSearchResponse.class);
```

	RestTemplate	RestClient	WebClient
스타일	구식 메서드	fluent · 권장	fluent · reactive
I/O	동기	동기	비동기(Mono/Flux)

유지보수	maintenance	활발	활발
------	-------------	----	----

외부 API 호출 시 반드시 고려할 것

- 타임아웃 — connectTimeout / readTimeout 명시 (기본 무제한이라 위험).
- 실패 처리 — 4xx/5xx · 쿼터 초과 · 네트워크 단절 각각 다른 정책. 이 프로젝트의 주변 맛집은 쿼터 초과 시 **키워드 검색으로 풀백**(커밋 0212ab1).
- 키 보안 — 절대 프론트엔드에 두지 말고 서버 프록시로 감싸기.
- 재시도 — idempotent 한 요청만, 지수 백오프.

13. 카카오페이 단건 결제 일반론

13-1. 2단계 결제 패턴

단계	주체	역할
① ready	가맹점 서버 → 카카오	주문 정보 등록 → tid + 결제창 URL 수신
② (사용자)	사용자 브라우저 → 카카오	결제창에서 결제 수단 선택·인증
③ approve	카카오 → 가맹점 콜백 → 가맹점 서버 → 카카오	pg_token 으로 최종 승인 → aid 발급

두 단계로 나누는 이유는 **승인 책임을 가맹점 서버가 명시적으로 지게 하기** 위함입니다. ready 만으로는 결제가 확정되지 않습니다.

13-2. 가맹점 서버가 관리해야 하는 식별자

- partnerOrderId — 가맹점이 발급. 결제 1건당 유일해야 함. (이 프로젝트: UUID 앞 12자).
- partnerUserId — 가맹점의 사용자 식별자.
- tid — 카카오가 발급하는 거래 ID. ready 응답에서 받아 저장 → approve 때 그대로 전송.
- aid — 승인 ID. approve 응답에서 받아 영수증/환불 추적용으로 저장.

13-3. 결제 상태 머신

READY → APPROVED (정상 결제 완료)
 → CANCELED (사용자가 결제창에서 취소)
 → FAILED (인증 실패 등)

결제는 금전이 오가므로 모든 상태 전이를 DB 에 기록하고, 외부 호출 실패와 DB 저장 실패가 **한쪽만 일어나지 않도록** 트랜잭션 경계를 신중히 설계해야 합니다.

14. 이메일 — JavaMailSender · SMTP

```
spring.mail.host=smtp.naver.com
spring.mail.port=465
spring.mail.username=${MAIL_USER}
spring.mail.password=${MAIL_PASS}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.ssl.enable=true
```

```
@Service
@RequiredArgsConstructor
public class EmailService {
    private final JavaMailSender mailSender;

    @Async // 호출 스레드를 막지 않음 (가임/대량 발송에 중요)
    public void sendWelcome(String to, String nickname) {
        MimeMessage msg = mailSender.createMimeMessage();
        MimeMessageHelper h = new MimeMessageHelper(msg, "UTF-8");
        h.setTo(to); h.setSubject("환영합니다");
        h.setText("<h1>" + nickname + "님, 환영해요</h1>", true /* HTML */);
        mailSender.send(msg);
    }
}
```

- SMTP 포트: 25(평문) · 465(SSL) · 587(STARTTLS).
- Gmail/Naver 는 앱 비밀번호가 별도 필요한 경우가 많습니다.
- 대량 발송은 한 번에 보내지 말고 큐 + 재시도 + 발송 결과 기록이 안전합니다.

15. 비동기와 스케줄링

15-1. @Async

```
@SpringBootApplication
@EnableAsync
public class StudyApplication { }

@Service
public class EmailService {
    @Async public void sendWelcome(...) { ... } // 호출 즉시 반환, 별도 스레드에서 실행
}
```

- 같은 클래스 내부 호출은 프록시를 거치지 않아 동작하지 않음(@Transactional 과 동일한 함정).
- 스레드 풀을 직접 정의([ThreadPoolTaskExecutor](#))해 코어 수/큐 크기를 통제하는 것이 좋습니다.

15-2. @Scheduled · Cron

```
@Component
@RequiredArgsConstructor
public class DailyMailScheduler {
    // 매일 KST 20:00
    @Scheduled(cron = "0 0 20 * * *", zone = "Asia/Seoul")
    public void sendDaily() { ... }
}
```

Cron 6필드(스프링) : `초 분 시 일 월 요일` — 표준 Unix cron 의 5필드와 다릅니다.

표현	의미
<code>0 0 20 * * *</code>	매일 20:00:00
<code>0 */10 * * * *</code>	10분마다
<code>0 0 9 * * MON-FRI</code>	평일 09:00

중복 실행 방지가 필요하면 이 프로젝트처럼 **마지막 발송 날짜를 파일/DB 에 기록**하거나, 멀티 인스턴스라면 ShedLock 같은 분산 락을 씁니다.

16. 예외 처리 — 전역 핸들러

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<MessageResponse> notFound(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new MessageResponse(e.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<MessageResponse> badRequest(MethodArgumentNotValidException e) {
        String msg = e.getBindingResult().getFieldErrors().stream()
            .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
            .collect(Collectors.joining("\n"));
        return ResponseEntity.badRequest().body(new MessageResponse(msg));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<MessageResponse> fallback(Exception e) {
        return ResponseEntity.status(500)
            .body(new MessageResponse(e.getClass().getSimpleName() + ": " + e.getMessage()));
    }
}
```

- 도메인 에러를 표준 예외에 의도적으로 매핑하면(예: 자원 없음 → `IllegalArgumentException`) 컨트롤러 코드가 단순해집니다.
- HTTP 상태 코드 매핑은 한 곳에서 일관되게: 400(검증) · 401(인증) · 403(권한) · 404(없음) · 409(충돌) · 500(서버).

17. OpenAPI / Swagger UI

springdoc-openapi 를 의존성에 넣으면 자동으로 컨트롤러를 스캔해 OpenAPI 문서를 생성합니다.

- `/v3/api-docs` — JSON 명세.
- `/swagger-ui/index.html` — 대화형 문서/시도.
- `@Operation`, `@Schema`, `@Parameter` 로 상세 설명을 보강.

18. 환경 설정

```
# application.properties
spring.datasource.url=jdbc:mariadb://localhost:3306/notice
spring.datasource.username=root
```

```
spring.datasource.password=${DB_PASSWORD}      # 환경변수 주입
spring.jpa.hibernate.ddl-auto=update

kakao.api.key=${KAKAO_REST_KEY}
kakaopay.cid=TC00NETIME
kakaopay.secret-key=${KAKAOPAY_SECRET_KEY}

app.admin.username=test11
app.admin.token=${ADMIN_TOKEN}
```

- `@Value("${kakao.api.key}")` 또는 `@ConfigurationProperties` 로 주입.
- 프로파일(`application-dev.properties` , `application-prod.properties`) 로 환경별 분리. 활성화: `--spring.profiles.active=prod`.
- 비밀 값은 코드/Git 에 절대 넣지 말고 환경변수 · Vault · AWS SSM 같은 외부 저장소로.

19. 학습 체크리스트

코드 흐름 한 번에 설명할 수 있는가?

- 로컬 회원가입에서 이메일 인증 → BCrypt 해시 → 세션 부여까지의 5단계.
- 카카오 OAuth 콜백이 도착했을 때 서버가 하는 4가지(code 재사용 차단, token 교환, 프로필 조회, upsert+세션).
- 카카오페이 ready / approve 가 분리된 이유와 각 단계에서 저장해야 하는 식별자.
- 외부 API 가 실패했을 때 사용자 경험을 깨지 않는 폴백 전략.
- 매일 20시 메일이 중복 발송되지 않게 하는 방법(파일 기록 vs 분산 락).

면접·코드 리뷰에서 자주 나오는 질문

- @Transactional 이 안 걸리는 대표 케이스 2가지? (자기 호출, private 메서드)
- JPA N+1 문제는 무엇이고, 이 코드에서는 어떻게 피했는가?
- BCrypt 가 같은 비밀번호에 다른 해시를 만드는 이유는?
- OAuth 의 state 파라미터가 왜 필요한가?
- CORS 와 CSRF 는 어떻게 다른가? 세션 인증에서 둘 다 신경 써야 하는 이유?
- JWT 대신 세션을 쓰면 멀티 인스턴스 운영 시 무엇이 문제가 되는가?
- DTO 와 엔티티를 분리하는 이유 3가지?

다음 단계로 공부하면 좋은 주제

- Spring Security 정식 도입 — 필터 체인, `SecurityFilterChain`, 인증/인가 분리.
- Refresh Token / JWT — 토큰 회전, 블랙리스트.
- Redis 세션 저장 — 다중 인스턴스.
- Flyway / Liquibase — DDL 마이그레이션 (ddl-auto=update 대체).
- Testcontainers — 실제 MariaDB 기반 통합 테스트.
- OpenTelemetry / Micrometer — 분산 추적, 메트릭.
- Resilience4j — 서킷 브레이커, 재시도, 타임아웃 (외부 API 안정성).

끝. 본 문서는 [backend-data-flow.pdf](#) 를 이해하기 위한 학습 자료입니다. 실제 코드와 차이가 발견되면 코드를 우선으로 하고 본 문서를 갱신 하세요.